

H A S H E D I N B Y D E L O I T T E
Engineering Learning & Development

Multi-Agent Systems Engineering

Mandatory Course Content — Modules 1 through 8

Document	MAS-TRAINING-001
Audience	Engineers with Python and basic LLM API experience
Author	Nabeel Qazi
Owner	Engineering Learning & Development
Version	1.0
Status	Internal Use

Document Control

Document Title	Multi-Agent Systems Engineering — Mandatory Course Content
Companion Docs	MAS-TRAINING-002 (Optional Content) · MAS-TRAINING-003 (Assignments) · MAS-TRAINING-004 (Evaluator Guide)
Total Study Time	~50–55 hours of guided study across 8 required modules
Format	Self-paced video courses, official docs, and hands-on labs
Prerequisites	Python 3.x, OpenAI API basics, LangChain fundamentals, SQL
Required Stack	LangGraph, FAISS, FastMCP (see MAS-TRAINING-003 for assignment specs)
Assessment	10 graded assignments (10 marks each, 100 marks total) — see MAS-TRAINING-003

1. Course Overview

Modern engineering teams increasingly rely on systems where multiple specialized AI agents collaborate to solve problems no single LLM call can handle alone. This program prepares engineers to design, build, and operate such multi-agent systems with confidence.

Across eight required modules and ten graded assignments, participants progress from the conceptual fundamentals of agent design through advanced orchestration patterns: function and tool calling, open agent protocols (MCP and A2A), data-grounded agents (RAG and database AI), canonical design patterns, observability, evaluation, and long-term agentic memory.

Why This Program?

- Single-LLM applications hit a ceiling once tasks require planning, tool use, retrieval, validation, and long-running state.
- Multi-agent patterns let teams decompose work the way a real organization does — with specialists, hand-offs, and review steps.
- Open standards (MCP via FastMCP, A2A) and production frameworks (LangGraph, AutoGen) are now mature enough for client delivery.
- Engineers who can fluently combine these tools deliver dramatically more value per engineering hour.

Who Should Attend

- Software engineers building LLM-powered features who are ready to move beyond single-prompt applications.
- Solution architects designing AI workflows for clients, especially those involving retrieval, databases, or multi-step automation.
- Technical leads who need a working understanding of orchestration patterns to make informed framework and architecture decisions.

Pacing Guide

Most engineers complete the required core in 7–9 weeks at 8–10 hours per week, with assignments submitted at the end of the 2-week intensive. Three optional modules (A, B, C) add 6–12 hours each and are shared post-training as a separate document (MAS-TRAINING-002).

2. Learning Outcomes

On successful completion of this program, participants will be able to:

- Define what an agent is in the modern LLM context and articulate the design trade-offs between single-agent and multi-agent solutions.
- Implement function and tool calling in OpenAI and LangGraph agents, with correct intent detection and graceful fallback behavior.
- Compare the leading multi-agent frameworks (LangGraph, AutoGen) and select the right one for a given problem.
- Build sequential, nested, and group-chat orchestration patterns and explain when each is appropriate.
- Apply the canonical agent design patterns — prompt chaining, routing, parallelization, orchestrator-workers, evaluator-optimizer, ReAct — and choose the simplest pattern that solves the problem.
- Design Retrieval-Augmented Generation (RAG) pipelines using FAISS that combine vector search with multi-agent reasoning.
- Apply the Model Context Protocol (MCP) using FastMCP to expose tools to any compliant agent host.
- Build database-aware agents that translate natural language to SQL safely against a real schema.
- Apply the A2A protocol to connect agents across organizational and runtime boundaries.
- Instrument agents with Langfuse or LangSmith and write LLM-as-judge evaluations using DeepEval.
- Design and implement an agentic memory architecture combining LangGraph state, LangMem, and Mem0.
- Evaluate and document agent systems with transcripts, logs, and clear orchestration diagrams.

3. Pre-requisites

This program assumes participants are comfortable writing Python and have basic exposure to LLM APIs. Where gaps exist, learners should remediate before Module 3.

Area	What you should already know
Python 3.x	Data types, functions, classes, modules, error handling. Virtual environments and pip. Experience with pandas, json, sqlite3, and requests.
Prompt Engineering	Crafting clear, context-rich prompts. Prompt iteration. Awareness of few-shot, chain-of-thought, and instruction-based techniques.
OpenAI API	Creating and using API keys. Invoking GPT-3.5/GPT-4 from Python. Understanding cost, rate limits, and basic error handling.
LangChain Basics	Core concepts: agents, tool usage, chains, and memory modules. Building a simple chained pipeline with an external tool or API.
Databases	Core SQL (select, join, group, where, limit). SQLite setup. Basic CRUD and schema design. Integrating Python with SQLite.
REST API Usage	HTTP verbs (GET, POST). Calling external APIs from Python (requests/httpx) and parsing JSON responses.
Vector Databases	Vector similarity search concepts. Experience with FAISS, Chroma, or Pinecone. Indexing, storing, and retrieving documents via embeddings.

4. Curriculum at a Glance

Eight required modules, progressing from foundations to production-grade agent systems. Three optional modules (A, B, C) are distributed separately as post-training extension material.

#	Module	Time	Focus
1	Introduction to Agents and Multi-Agent Systems	~2 hrs	Foundations and vocabulary
2	Multi-Agent System Design	~2 hrs	Function and tool calling
3	Multi-Agent Frameworks & Deep Dives	~10 hrs	AutoGen, LangGraph
4	Agent Protocols: MCP & A2A	~9.5 hrs	FastMCP, A2A protocol
5	Data-Grounded Agents: RAG & Database AI	~11 hrs	RAG, FAISS, SQL agents
6	Agent Design Patterns	~7 hrs	Collaborating agents, patterns
7	Agent Observability, Evaluation & Testing	~6 hrs	Tracing, evals, LLM-as-judge
8	Agentic Memory	~5 hrs required	Short, long-term, LangMem, Mem0

Required stack across all assignments: LangGraph · FAISS · FastMCP. See MAS-TRAINING-003 for full assignment specifications.

Module 1 — Introduction to Agents and Multi-Agent Systems

This opening module establishes the vocabulary and mental model used throughout the program. Engineers should leave Module 1 able to explain what an agent is, what makes a system 'multi-agent', and recognize where these patterns deliver real value.

Estimated time: ~2 hours

Lesson 1.1 — What is an Agent?

Learning Outcomes

- Define an 'agent' in the AI context.
- Identify real-world agent use cases and recognize where autonomous agents outperform single LLM calls.

Resources

- [YouTube — What is an AI Agent?](#) (30 mins)
- [DeepLearning.AI Short Course — AI Agents in LangGraph](#) (1.5 hours)

Lesson 1.2 — What is a Multi-Agent System (MAS)?

Learning Outcomes

- Explain multi-agent system concepts and the key differences from single-agent designs.
- Identify real-world applications for MAS in retrieval, automation, and decisioning.

Resources

- [YouTube — Multi-Agent Introduction](#) (9 mins)

Module 2 — Multi-Agent System Design

Once participants understand what an agent is, the next step is learning how agents reach beyond their own reasoning to take action. Function and tool calling is the mechanism that turns a chat-completion model into an autonomous worker.

Estimated time: ~2 hours

Lesson 2.1 — Function and Tool Calling

Learning Outcomes

- Understand function/tool calls in agent orchestration — how an LLM emits a typed call payload and receives a result.
- Design tool schemas that LLMs can reliably select and populate.
- Recognize practical scenarios for tool usage: calculation, retrieval, side effects.

Resources

- [DeepLearning.AI — Functions, Tools and Agents with LangChain](#) (2 hours)

Module 3 — Multi-Agent Frameworks & Deep Dives

Module 3 first surveys the multi-agent framework landscape so participants can make informed choices, then dives deep into the two core orchestration frameworks used throughout the program: AutoGen and LangGraph.

Estimated time: ~10 hours total

Lesson 3.1 — Overview of Popular Frameworks

Learning Outcomes

- Survey the commonly used multi-agent frameworks: LangChain, LangGraph, AutoGen, CrewAI, ADK.
- Compare their orchestration philosophies, primary use-cases, and ecosystem maturity.

Resources

- [YouTube — Multi-Agent Frameworks Compared](#) (8 mins)

Lesson 3.2 — AutoGen Framework

Learning Outcomes

- Install and set up AutoGen.
- Build basic-to-advanced multi-agent workflows using AutoGen group chat patterns.
- Understand the roles of UserProxy, AssistantAgent, and GroupChatManager.

Resources

- [Udemy — AutoGen Agent Systems](#) (3.5 hours)

Lesson 3.3 — LangGraph Framework

LangGraph is the primary framework for all assignments in this program. Every assignment uses LangGraph StateGraph — not vanilla LangChain AgentExecutor.

Learning Outcomes

- Understand LangGraph concepts: state graphs, nodes, edges, and conditional routing.
- Design and implement advanced agent workflows with explicit state management.
- Use LangGraph Studio as the visual IDE for debugging and inspecting graph runs.

Resources

- [Udemy — LangGraph in Action](#) (3.5 hours)
- [Udemy — LangGraph \(supplementary, with LLM\)](#) (3 hours)

Module 4 — Agent Protocols: MCP & A2A

Modern agent systems need standardized ways to talk to tools, data, and each other. This module covers the two open protocols that are rapidly becoming the infrastructure layer of production agent deployments: the Model Context Protocol (MCP) for connecting agents to external capabilities, and Google's Agent-to-Agent (A2A) protocol for cross-system agent communication.

Estimated time: ~9.5 hours

Lesson 4.1 — Model Context Protocol (MCP) with FastMCP

This program uses FastMCP (pip install fastmcp) — the Python implementation of MCP. Do not use raw MCP configuration or the TypeScript implementation for assignments.

Learning Outcomes

- Understand MCP's client-server architecture: hosts, clients, and servers.
- Integrate MCP servers into multi-agent workflows built with LangGraph.
- Build FastMCP servers that expose tools and resources to any compliant agent host.
- Reason about MCP's role in the broader protocol stack alongside A2A.

Resources

- [Udemy — Model Context Protocol \(full course\)](#) (5.5 hours)

Lesson 4.2 — Google Agent-to-Agent (A2A) Protocol

Learning Outcomes

- Understand the fundamentals and architecture of Google's A2A protocol.
- Deploy and host A2A agents using Google Cloud Run.
- Understand how A2A and MCP are complementary: MCP connects agents to tools; A2A connects agents to agents.
- Explore real-world use cases and the current A2A upgrade path.

Resources

- [Google A2A Official Docs](#) (2–3 hours)
- [Google Cloud Blog — A2A Protocol Upgrade](#) (15 mins)
- [Google Cloud Run — Host A2A Agents](#) (1 hour)
- [YouTube — A2A Protocol: Introduction & Concepts](#) (8 mins)
- [YouTube — Building and Deploying A2A Agents on Cloud Run](#) (25 mins)

Module 5 — Data-Grounded Agents: RAG & Database AI

Production agents must stay grounded in real, current knowledge rather than relying solely on model weights. This module covers the two dominant grounding strategies: Retrieval-Augmented Generation (RAG) for unstructured knowledge, and natural-language-to-SQL agents for structured databases.

Estimated time: ~11 hours

Lesson 5.1 — Introduction to RAG

FAISS is the required vector store for all RAG-based assignments. Do not substitute Chroma or Pinecone in submitted code.

Learning Outcomes

- Explain why and when to use RAG versus fine-tuning or longer context windows.
- Connect RAG with multi-agent orchestration — where retrieval fits in a multi-step pipeline.
- Set up a basic end-to-end RAG pipeline: chunk, embed, index, retrieve, generate — using FAISS.

Resources

- [Udemy — LLM Engineering, Section 5: Mastering RAG](#) (2 hours)
- [DeepLearning.AI — Building Applications with Vector Databases](#) (1.5 hours)

Lesson 5.2 — Advanced RAG with Agents

Learning Outcomes

- Design advanced RAG workflows: re-ranking, hybrid search, query expansion, corrective RAG.
- Integrate FAISS retrievers into LangGraph multi-agent pipelines where specialist agents retrieve then synthesize.
- Apply advanced vector database techniques for production-scale retrieval.

Resources

- [Udemy — RAG Masterclass](#) (5 hours)

Lesson 5.3 — Database AI Agents

Learning Outcomes

- Build agents for NL → SQL translation, query analysis, and result summarization.
- Apply schema introspection patterns to keep generated SQL grounded in the actual database schema.
- Implement guardrails and validation to prevent destructive or unintended queries.
- Combine RAG (FAISS) and SQL agents in a single LangGraph workflow for mixed-data-source questions.

Resources

- [Udemy — Database Agents](#) (2.5 hours)

Module 6 — Agent Design Patterns

This module gives engineers the mental models to choose orchestration patterns deliberately. It opens with a practical hands-on study of collaborating agents in action, then moves to Anthropic's canonical pattern vocabulary — the single most-cited reference in agent engineering.

Core principle: the most successful agent implementations use simple, composable patterns. Reach for the simplest pattern that solves the problem; add complexity only when it pays.

Estimated time: ~7 hours

Lesson 6.1 — Collaborating Agents in Practice

Learning Outcomes

- Understand the dynamics of collaboration between multiple agent models in real pipelines.
- Implement autonomy and self-direction in agent orchestration.
- Recognize common collaboration anti-patterns and how to avoid them.

Resources

- [Udemy — LLM Engineering, Section 8: Collaborating Agents](#) (4 hours)

Lesson 6.2 — Workflows vs. Agents: When Does an LLM Need to Drive?

Learning Outcomes

- Distinguish workflows (predefined paths, programmatic orchestration) from agents (LLM-directed, dynamic tool selection).
- Reason about latency, cost, reliability, and flexibility trade-offs.
- Recognize when a workflow beats a fully autonomous agent.

Resources

- [Anthropic — Building Effective Agents](#) (core reading)

Lesson 6.3 — The Augmented LLM

Learning Outcomes

- Treat the augmented LLM — model + retrieval + tools + memory — as the basic unit of agent design.
- Tailor each augmentation to the use case rather than reaching for every capability.
- Evaluate when MCP (FastMCP) is the right integration surface for tools and resources.

Resources

- [Anthropic — Building Effective Agents \(Augmented LLM section\)](#) (core reading)

Lesson 6.4 — Pattern 1: Prompt Chaining

Learning Outcomes

- Decompose a single hard task into a sequence of simpler LLM calls.
- Insert programmatic checks (gates) between stages to catch errors early.
- Recognize when chaining beats a single, larger prompt.

Resources

- [Anthropic — Building Effective Agents \(Prompt Chaining\)](#) *(core reading)*
- [Anthropic Cookbook — Prompt Chaining implementation](#) *(code)*

Lesson 6.5 — Pattern 2: Routing

Learning Outcomes

- Build classifier-style first hops that direct queries to specialized downstream chains.
- Decide between LLM-based and embedding-based routers.
- Apply routing to triage flows, escalation tiers, and complexity-based model selection.

Resources

- [Anthropic — Building Effective Agents \(Routing section\)](#) *(core reading)*

Lesson 6.6 — Pattern 3: Parallelization

Learning Outcomes

- Run the same task multiple times for diverse outputs (voting) — useful for code review and content moderation.
- Break a task into independent subtasks that run concurrently (sectioning).
- Aggregate parallel results programmatically rather than letting an LLM decide.

Resources

- [Anthropic — Building Effective Agents \(Parallelization section\)](#) *(core reading)*

Lesson 6.7 — Pattern 4: Orchestrator-Workers

Learning Outcomes

- A central LLM dynamically breaks down tasks, dispatches them to worker LLMs, and synthesizes results.
- Compare orchestrator-workers to fixed sectioning: the orchestrator decides at runtime what subtasks exist.
- Apply the pattern to research agents, code-modification agents, and complex automations.

Resources

- [Anthropic — Building Effective Agents \(Orchestrator-Workers\)](#) *(core reading)*

Lesson 6.8 — Pattern 5: Evaluator-Optimizer

Learning Outcomes

- Pair a producer LLM with an evaluator LLM that provides feedback in a loop.
- Apply to literary translation, complex search, and code refinement tasks.
- Define stop conditions to avoid quality plateaus and runaway loops.

Resources

- [Anthropic — Building Effective Agents \(Evaluator-Optimizer\)](#) *(core reading)*

Lesson 6.9 — Additional Patterns: ReAct, Plan-and-Execute, Reflection

Learning Outcomes

- ReAct (Reasoning + Acting): interleave reasoning and tool-use in a Thought → Action → Observation loop.
- Plan-and-Execute: separate planning (large model, full task view) from execution (smaller model, single step).
- Reflection: a generator LLM produces output and a reflector LLM critiques it, iterating until quality criteria are met.

Resources — ReAct

- [IBM — What is a ReAct Agent?](#) *(20 mins)*
- [Machine Learning Mastery — Building ReAct Agents with LangGraph](#) *(45 mins)*

Resources — Plan-and-Execute

- [LangGraph — Plan-and-Execute official tutorial](#) *(45 mins)*

Resources — Reflection

- [LangChain Blog — Reflection Agents](#) *(30 mins)*

Lesson 6.10 — Choosing the Right Pattern

Learning Outcomes

- Apply a decision framework: start simple, measure, add complexity only when it delivers measurable value.
- Design the agent-computer interface (ACI): tool documentation, transparency, and human oversight.
- Avoid common anti-patterns: premature multi-agent designs, over-abstracted frameworks, and hidden state.

Resources

- [Anthropic — Building Effective Agents \(Agents in Practice\)](#) *(core reading)*
- [Medium — Five Agentic Workflow Patterns](#) *(20 mins)*

Module 7 — Agent Observability, Evaluation & Testing

Production agents fail in ways single-prompt LLMs do not — drifting tools, runaway loops, ballooning costs, regressions on edge cases. This module combines instrumentation (observability) with quality measurement (evaluation and testing). Together they form the production quality loop.

Estimated time: ~6 hours

Lesson 7.1 — Why Agent Observability Is Different

Learning Outcomes

- Recognize why multi-step agent traces require richer instrumentation than single-call LLM logs.
- Identify the failure modes specific to long agent runs: tool drift, sub-agent confusion, retry storms, cost overruns.
- Adopt OpenTelemetry as the emerging standard for vendor-agnostic agent telemetry.

Resources

- [Langfuse — AI agent observability with LangGraph, AutoGen, OpenAI Agents, CrewAI](#) (30 mins)

Lesson 7.2 — Tracing with Langfuse

Learning Outcomes

- Set up the open-source Langfuse stack locally or in production.
- Instrument LangChain, AutoGen, LangGraph, and OpenAI Agents applications.
- Track cost, latency, and token usage per trace and per user.

Resources

- [Langfuse — Observability overview \(official docs\)](#) (1 hour)

Lesson 7.3 — Tracing with LangSmith

Learning Outcomes

- Use LangSmith for LangChain- and LangGraph-native observability.
- Apply LangGraph Studio as the visual agent IDE for stepwise debugging.
- Capture annotation queues and datasets directly from production traces.

Resources

- [LangSmith — Observability \(official docs\)](#) (1 hour)

Lesson 7.4 — Foundations of Agent Evaluation

Learning Outcomes

- Distinguish the three evaluation layers: reasoning quality, tool selection correctness, end-task completion.
- Compare offline evaluation (static datasets) with online evaluation (sampled production traffic).
- Recognize why naive success rates mislead; use progress rate and repetition rate to debug agent behavior.

Resources

- [DeepEval — AI Agent Evaluation guide](#) (1 hour)

Lesson 7.5 — LLM-as-Judge: Building Reliable Auto-Evaluators

Learning Outcomes

- Design rubrics: binary or 3-point scales beat 1–10 because they have lower variance.
- Choose a judge model at least as capable as the system under test.
- Apply majority-vote patterns (e.g. 2-of-3 with threshold 0.66) to reduce judge variance.
- Treat candidate output as untrusted data — guard the judge prompt against prompt injection.

Resources

- [Confident AI — G-Eval definitive guide](#) (30 mins)
- [Promptfoo — LLM-as-a-Judge guide](#) (1 hour)

Lesson 7.6 — DeepEval: Pytest-Style LLM Testing

Learning Outcomes

- Write LLM unit tests with metrics: G-Eval, task completion, answer relevancy, hallucination, contextual relevancy.
- Integrate evals into CI so every pull request is graded automatically.
- Run regression tests when changing models, prompts, or tools.

Resources

- [DeepEval \(open source, MIT\) — GitHub](#) (code)

Lesson 7.7 — Promptfoo: Red-Teaming and Safety Validation

Learning Outcomes

- Build adversarial test suites and run red-teaming campaigns against agents.
- Combine deterministic assertions with LLM-as-judge to balance speed and depth.
- Use eval caching to keep CI runs fast and reproducible.

Resources

- [Promptfoo — LLM-as-a-Judge guide \(includes red-teaming section\)](#) (1 hour)

Lesson 7.8 — Evaluating RAG Specifically

Learning Outcomes

- Apply RAGAS metrics: faithfulness, answer relevancy, context precision, context recall.
- Diagnose whether retrieval or generation is the failure point and measure the right layer.
- Build golden datasets from real production traces rather than synthetic-only data.

Resources

- [RAGAS — Metrics documentation](#) (30 mins)
- [Langfuse — Evaluating RAG with RAGAS](#) (45 mins)

Lesson 7.9 — The Production Quality Loop

Learning Outcomes

- Operate the full loop: failing traces → curated dataset → improved prompt or model → re-run evals → ship.
- Sample production traffic for online evaluation and monitor quality degradation over time.
- Implement cost guards and circuit breakers so a runaway agent cannot exhaust budget or quota.

Resources

- [Langfuse — Observability overview \(covers production monitoring\)](#) (1 hour)

Module 8 — Agentic Memory

Context windows are not memory. Even a 1M-token window is uneconomical for daily agents and suffers from the lost-in-the-middle problem. Production agents need a deliberate memory architecture: short-term thread state, episodic session memory, and long-term semantic and procedural memory.

Structure: Lessons 8.1–8.3 and 8.7 are required (covered in this document). Lessons 8.4–8.6 (Neo4j Knowledge Graphs, Letta, OpenClaw) are optional self-directed extensions distributed separately in MAS-TRAINING-002.

Estimated time: ~5 hours required; ~3 hours additional for optional lessons 8.4–8.6 (separate document)

Lesson 8.1 — Memory Fundamentals: Why Context ≠ Memory

Learning Outcomes

- Articulate why even large context windows are not a substitute for memory — cost, latency, and the lost-in-the-middle phenomenon.
- Distinguish four memory types: working/short-term, episodic, semantic, and procedural.
- Distinguish hot-path memory writes (during the response) from background writes (after).
- Recognize when memory is genuinely needed versus when retrieval alone is enough.

Resources

- [DeepLearning.AI — Long-Term Agentic Memory with LangGraph](#) (1 hour)
- [LangGraph — Memory overview docs](#) (30 mins)

Lesson 8.2 — Short-Term and Thread Memory with LangGraph

Learning Outcomes

- Implement thread-scoped state with LangGraph checkpoints.
- Use the BaseStore interface for cross-thread memory persistence.
- Apply summarization and compression to keep threads under context limits.

Resources

- [Atlan — Long-Term Memory in LangChain Agents \(5 approaches with benchmarks\)](#) (1 hour)

Lesson 8.3 — Long-Term Memory with LangMem and Mem0

Learning Outcomes

- Use the LangMem SDK to extract semantic, episodic, and procedural memory from conversations.
- Use Mem0 for managed three-tier (user/session/agent) memory backed by vectors and graphs.
- Apply long-term memory to personalize outputs across sessions.

Resources

- [LangChain blog — LangMem SDK launch](#) (30 mins)
- [Mem0 paper \(91% token reduction benchmark\)](#) (research)
- [Mastering Long-Term Agentic Memory with LangGraph](#) (45 mins)

Lesson 8.4 — Knowledge Graphs for Agent Memory with Neo4j [Optional — see MAS-TRAINING-002]

This lesson is optional. Full content including learning outcomes and all resources is provided in MAS-TRAINING-002 (Optional Course Content), distributed post-training.

Lesson 8.5 — MemGPT / Letta: LLM-as-OS Memory [Optional — see MAS-TRAINING-002]

This lesson is optional. Full content is provided in MAS-TRAINING-002 (Optional Course Content), distributed post-training.

Lesson 8.6 — OpenClaw: Self-Hosted Multi-Agent Memory [Optional — see MAS-TRAINING-002]

This lesson is optional. Full content is provided in MAS-TRAINING-002 (Optional Course Content), distributed post-training.

Lesson 8.7 — Choosing a Memory Architecture

Learning Outcomes

- Use the LongMemEval and LoCoMo benchmarks to read published memory numbers honestly.
- Apply a decision tree: Mem0 (managed personalization), Neo4j (relationship-aware knowledge graph), LangMem (LangChain-native), Letta (long-running autonomous), local-first stacks (compliance).
- Plan for memory governance: contradicting memories, decay, confidence scoring, and audit trails.

Resources

- [DEV — 5 AI Agent Memory Systems Compared \(LoCoMo benchmarks\)](#) (30 mins)
- [Atlan — Best AI Agent Memory Frameworks 2026](#) (30 mins)

Appendix A — Glossary

Term	Definition
Agent	An LLM-driven component that perceives input, plans, takes actions via tools, and returns a result. Typically autonomous within its scope.
Multi-Agent System (MAS)	A system in which two or more agents collaborate, compete, or specialize to solve a problem. Patterns include sequential, nested, and group chat.
Tool / Function Calling	Mechanism by which an LLM invokes structured external capabilities by emitting a typed function-call payload.
LangGraph	Graph-based extension of LangChain for stateful, cyclic, and conditionally-routed agent workflows. Used for all assignments.
AutoGen	Microsoft framework for conversational multi-agent orchestration, including group chat patterns.
FastMCP	Python implementation of the Model Context Protocol. Used for MCP-based assignments.
MCP	Model Context Protocol: open protocol for exchanging structured context, tools, and resources between LLM hosts and external systems.
A2A	Agent-to-Agent Protocol: open protocol standardizing how independent agents discover and communicate across systems.
RAG	Retrieval-Augmented Generation: pattern that grounds LLM output in retrieved documents from a vectorstore before generation.
FAISS	Facebook AI Similarity Search: the required vector store for all RAG-based assignments. Open-source, local-first.
RAGAS	Open-source evaluation framework for RAG pipelines. Provides metrics including faithfulness, answer relevancy, context precision, and context recall.
Vector Store	Database optimised for similarity search over high-dimensional embeddings. FAISS is used in this program.
LangChain	Python/JS framework for building LLM applications using composable chains, agents, tools, retrievers, and memory.
LangSmith	Observability and debugging platform for LangChain/LangGraph applications.
Langfuse	Open-source observability platform for LLM applications; supports LangGraph, AutoGen, OpenAI.
LangMem	LangChain SDK for extracting and managing semantic, episodic, and procedural memory from LLM conversations.
Mem0	Managed memory platform providing three-tier (user/session/agent) memory backed by vector and graph stores.
Sequential Pattern	Multi-agent orchestration where each agent's output becomes the next agent's input in a fixed order.
Nested Pattern	Pattern where one agent invokes another agent within its own reasoning step, often recursively.

Term	Definition
Group Chat Pattern	Pattern where multiple agents share a common conversation, see each other's messages, and converge through dialogue.
ReAct	Reasoning + Acting: agent pattern that interleaves Thought → Action → Observation loops.
Plan-and-Execute	Agent pattern that separates planning (large model decomposes the full task) from execution (smaller model runs individual steps).
Evaluator-Optimizer	Pattern where one LLM produces output and another evaluates and critiques it in a feedback loop until quality criteria are met.
LLM-as-Judge	Evaluation technique in which an LLM scores or critiques the output of another LLM according to a defined rubric.